

Portfolio Project Documentation

Multi Agent Researcher

Deep-dive technical documentation — 14 sections



Generated April 07, 2026

Introduction

Readme

multi-agent-researcher — Documentation Index

Deep documentation for the Multi-Agent Research Assistant project.

Each file covers one component or concept in full — line-by-line explanations, diagrams, the "why" behind every design decision, and interview prep.

Files

#	File	What it covers
01	architecture.md	Big-picture design, data flow, all 5 services, design decisions
02	state.md	<code>ResearchState</code> TypedDict — field lifecycle, why TypedDict
03	graph.md	LangGraph state machine — nodes, edges, <code>Send</code> , fan-out/fan-in
04	a2a-client.md	A2A HTTP client — httpx, polling loop, timeout handling
05	base-a2a-server.md	Shared A2A server factory — task lifecycle, <code>asyncio.create_task</code>
06	web-search-agent.md	Web Search Agent — Tavily API, agent card, result formatting
07	rag-agent.md	RAG Agent — <code>sys.path</code> surgery, <code>asyncio.to_thread</code> , <code>answer_question</code>
08	synthesis-agent.md	Synthesis Agent — Claude, JSON parsing, markdown report generation
09	mcp-server.md	MCP Server — stdio transport, lazy loading, 3 tools
10	api-main.md	FastAPI Orchestrator — <code>/research</code> , <code>/upload</code> , <code>/documents</code>

11	ui-app.md	Streamlit UI — upload flow, research submission, result rendering
12	concepts.md	Concepts glossary — every term used across the project
13	interview-prep.md	25 interview Q&A covering all key skills

Quick start

```
# 5 terminals
python agents/web_search/agent.py      # port 8001
python agents/rag_agent/agent.py       # port 8002
python agents/synthesis/agent.py       # port 8003
uvicorn api.main:app --reload           # port 8000
streamlit run ui/app.py                 # port 8501
```

Required `.env` keys: `ANTHROPIC_API_KEY`, `TAVILY_API_KEY`

Section 1

Architecture

Architecture Deep Dive

The Big Picture

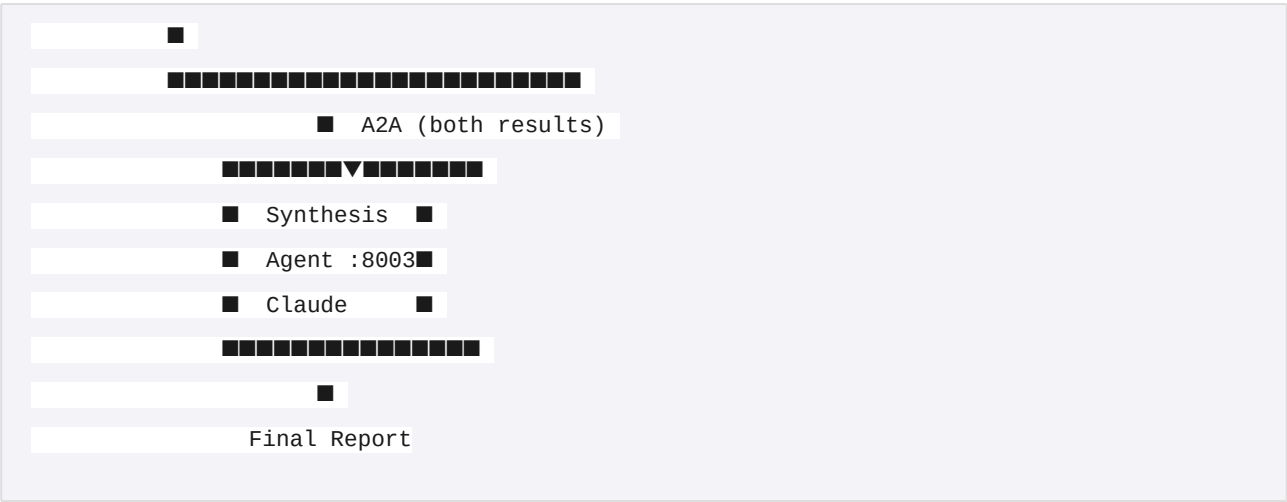
This project is a **multi-agent research assistant**. Instead of asking a single LLM to answer a research question from memory, it:

1. **Plans** — decomposes the question into targeted sub-queries
2. **Searches in parallel** — one agent hits the live web, another searches uploaded documents
3. **Synthesises** — a third agent merges both result sets into a polished markdown report

This mirrors how a real research team works: a manager breaks the work into tasks, junior researchers work in parallel, and a senior analyst writes the final report.

The 5 Services

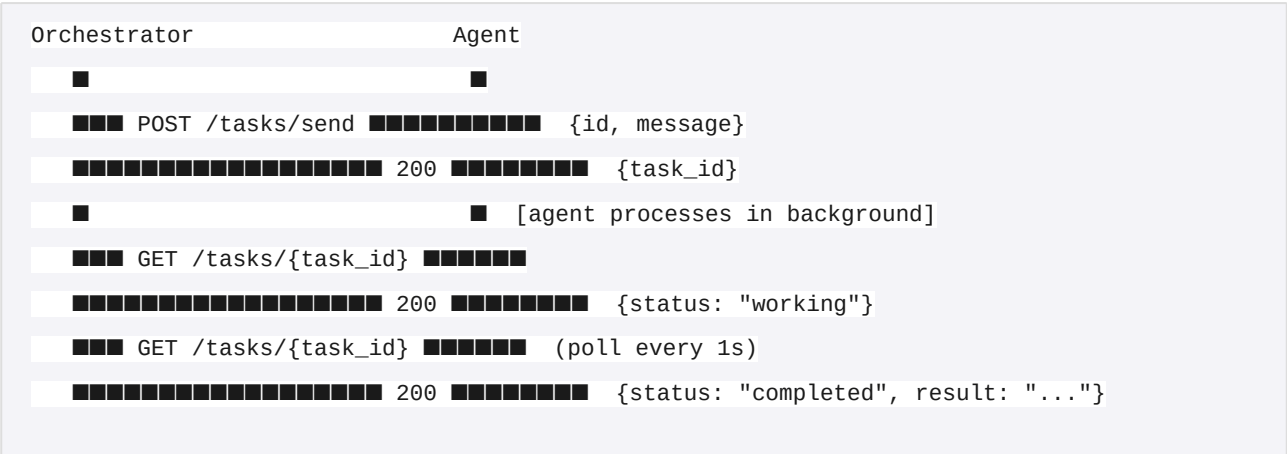




Two Protocols Layered Together

A2A (Agent-to-Agent)

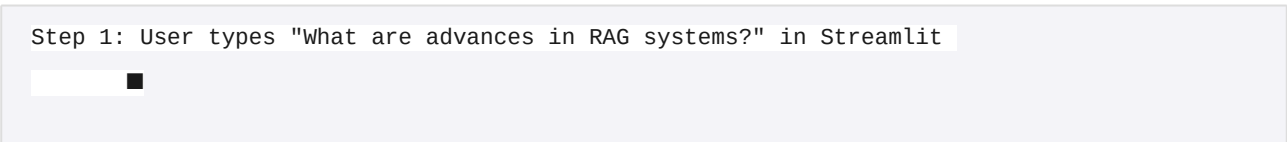
Used for **HTTP communication between services**. Each agent is its own FastAPI server with three standard endpoints. The orchestrator sends tasks and polls for results.



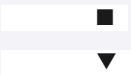
MCP (Model Context Protocol)

Used by the RAG Agent to call rag-doc-qa services. The RAG agent does **not** use MCP at runtime in this implementation — it imports rag-doc-qa directly. The MCP server ([mcp_server/server.py](#)) is a standalone tool for connecting Claude Desktop or other MCP clients to the same knowledge base.

Data Flow — Full Research Request



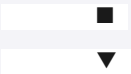
Step 2: POST /upload (multipart form data) → Orchestrator :8000



Step 3: api/main.py calls process_upload() from rag-doc-qa

- This runs in asyncio.to_thread() (non-blocking)
- process_upload: loads file → chunks → embeds → stores in ChromaDB

Step 4: Returns {doc_id, num_chunks, num_pages}



Step 5: Streamlit shows "Uploaded X – 47 chunks indexed."

Note: The `chroma_db/` folder is local and gitignored. Documents persist across restarts but are machine-specific.

Why This Architecture?

Design Decision	Reason
5 separate services	Shows real microservice patterns; each agent is independently deployable
A2A over HTTP	Standard protocol — any agent can be replaced without touching the orchestrator
LangGraph for orchestration	State is tracked in a TypedDict; parallel branches are first-class
<code>Send</code> for parallelism	LangGraph's built-in fan-out; no manual threading needed
<code>asyncio.to_thread</code> everywhere	All blocking calls (Anthropic SDK, ChromaDB) run off the event loop
Reuse rag-doc-qa	Code reuse via sys.path injection — no duplication, same ChromaDB
MCP server (separate)	Shows you can build MCP tools; useful for Claude Desktop integration

Service Responsibilities Summary

Service	Port	Technology	Responsibility
Orchestrator	8000	FastAPI + LangGraph	Receives query, runs graph, returns report
Web Search Agent	8001	FastAPI + Tavily	Live web search, formats snippets
RAG Agent	8002	FastAPI + rag-doc-qa	Knowledge base Q&A with citations
Synthesis Agent	8003	FastAPI + Claude	Merges results into markdown report
Streamlit UI	8501	Streamlit	User interface — upload + query
MCP Server	stdio	MCP SDK	Exposes RAG tools to Claude Desktop

Section 2

State

orchestrator/state.py — ResearchState

What this file does

Defines the **shared state** that flows through every node in the LangGraph graph. Think of it as the "clipboard" the entire pipeline reads from and writes to.

Full File: [orchestrator/state.py](#)

```
from typing import TypedDict, Optional

class ResearchState(TypedDict):
    query: str
    sub_tasks: list[str]
    web_results: str
    rag_results: str
    synthesis_input: str
    final_report: str
    status: str
    error: Optional[str]
```

Line-by-Line Explanation

`from typing import TypedDict, Optional`

- `TypedDict` — a special dict subclass from Python's `typing` module where every key has a declared type. The dict is still a regular Python dict at runtime, but type checkers (mypy, pyright) and LangGraph's internal validation treat the types as contracts.
- `Optional[str]` — shorthand for `str | None`. Used for the `error` field because it only has a value when something goes wrong; it's `None` during normal execution.

`class ResearchState(TypedDict):`

Inheriting from `TypedDict` tells Python (and LangGraph) that every instance of this class must be a dict with exactly these keys and their declared types.

Why TypedDict instead of a Pydantic model or a dataclass?

LangGraph requires state to be a `TypedDict`. Internally, LangGraph merges partial state updates from each node (e.g., `{"web_results": "..."}`) into the shared state dict. TypedDict allows that partial-update pattern naturally — you don't need to provide all fields when a node returns, just the ones it changed.

If you used a Pydantic model you'd need to return the entire model every time; that breaks the parallel fan-out pattern where two nodes each write different fields simultaneously.

Field-by-Field

`query: str`

The original research question submitted by the user. Set once in `api/main.py` when the initial state is built. Never modified after that — it's the "source of truth" for all downstream nodes.

`sub_tasks: list[str]`

Populated by `plan_node`. A two-element list:

- `sub_tasks[0]` — the web search query (sent to the Web Search Agent)
- `sub_tasks[1]` — the RAG query (sent to the RAG Agent)

Using a list (rather than two separate fields) keeps the plan extensible — a future planner could decompose into 3 or 4 sub-tasks without changing the state schema.

`web_results: str`

Raw text returned by the Web Search Agent. Contains the formatted snippets from Tavily. Populated by `web_search_node`, read by `synthesize_node`.

`rag_results: str`

Raw text returned by the RAG Agent. Contains the answer + source citations from ChromaDB + Claude. Populated by `rag_search_node`, read by `synthesize_node`.

Critical design note: `web_results` and `rag_results` are SEPARATE fields on purpose. Both `web_search_node` and `rag_search_node` run in parallel — if they both wrote to a single `results` field, LangGraph would see two simultaneous writes to the same key and raise:

```
INVALID_CONCURRENT_GRAPH_UPDATE
```

By assigning each node its own output key, the parallel writes never conflict.

`synthesis_input: str`

A JSON string that bundles `query + web_results + rag_results` into a single payload for the Synthesis Agent. Built by `synthesize_node` before calling the agent. Stored in state for debugging — you can inspect what was actually sent to the synthesis agent if the report looks wrong.

`final_report: str`

The finished markdown research report returned by the Synthesis Agent. This is the field the API and UI care about most. Populated by `synthesize_node`.

`status: str`

A human-readable label for the current execution phase. Updated by `plan_node` ("planning") and `synthesize_node` ("synthesizing"). In the UI, if `status != "completed"`, the value is shown as an info banner.

Note: parallel nodes (`web_search_node`, `rag_search_node`) do NOT update `status` — updating the same field from two concurrent nodes would cause the same `INVALID_CONCURRENT_GRAPH_UPDATE` error.

`error: Optional[str]`

Populated only if an unhandled exception occurs. `None` during normal execution. Not currently used in the graph nodes directly — the API layer catches exceptions and raises HTTP 500 — but it's a forward-compatible slot for per-node error reporting.

How State Flows Through the Graph

Initial state (set in `api/main.py`):

```
{
  "query": "What are the latest advances in RAG?",
  "sub_tasks": [],
  "web_results": "",
  "rag_results": "",
  "synthesis_input": "",
  "final_report": "",
  "status": "starting",
  "error": None,
}
```

After `plan_node`:

```
{
```

```

...
"sub_tasks": ["latest RAG systems 2024", "RAG retrieval techniques"],
"status": "planning",
}

After web_search_node (runs in parallel with rag_search_node):
{
...
"web_results": "Web search results for: ...\n\n1. Title\n  URL: ...",
}

After rag_search_node (runs in parallel with web_search_node):
{
...
"rag_results": "Answer: ...\n\nSources:\n 1. ...",
}

After synthesize_node:
{
...
"synthesis_input": "{\"query\": ..., \"web_results\": ..., \"rag_results\": ...}",
"final_report": "## Summary\n...\n## Key Findings\n...",
"status": "synthesizing",
}

```

LangGraph merges each node's return dict into the shared state automatically. Nodes return only the fields they changed — LangGraph handles the merge.

Section 3

Graph

orchestrator/graph.py — LangGraph State Machine

What this file does

Defines and compiles the **LangGraph research pipeline** — the state machine that decomposes a user question, dispatches to specialist agents, and synthesises the final report.

Execution Flow

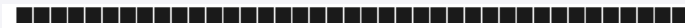
START



plan_node → Claude Haiku decomposes query → 2 sub-tasks



dispatch_parallel_searches() returns [Send(...), Send(...)]



web_search_node

rag_search_node

(A2A → :8001)

(A2A → :8002)



fan-in (LangGraph waits for both)



synthesize_node

→ A2A → :8003 → Claude Sonnet



END

Imports

```
import json
import logging
import os
from typing import Any

from langchain_anthropic import ChatAnthropic
```

```

from langchain_core.messages import HumanMessage, SystemMessage
from langgraph.constants import END, START, Send
from langgraph.graph import StateGraph

from orchestrator.a2a_client import send_task
from orchestrator.state import ResearchState

```

- `ChatAnthropic` — LangChain wrapper around the Anthropic Python SDK. Provides a `.ainvoke()` async method that returns a LangChain `AIMessage`.
- `HumanMessage`, `SystemMessage` — LangChain message types. These map to the `{"role": "user"/"system", "content": "..."} format Claude expects.`
- `END`, `START` — LangGraph's sentinel constants for the graph entry and exit nodes.
- `Send` — LangGraph's parallel dispatch primitive. Returns a `Send` object from an edge function to tell LangGraph to run a specific node with a specific state snapshot.
- `StateGraph` — the main class for building LangGraph graphs.

Constants

```

WEB_SEARCH_AGENT_URL = os.getenv("WEB_SEARCH_AGENT_URL", "http://localhost:8001")
RAG_AGENT_URL         = os.getenv("RAG_AGENT_URL",         "http://localhost:8002")
SYNTHESIS_AGENT_URL   = os.getenv("SYNTHESIS_AGENT_URL",   "http://localhost:8003")
PLANNER_MODEL         = os.getenv("PLANNER_MODEL",         "claude-haiku-4-5-20251001")

```

Agent URLs are read from environment variables with localhost defaults. In Docker, you'd set `WEB_SEARCH_AGENT_URL=http://web_search:8001` to use Docker's internal DNS.

`claude-haiku-4-5-20251001` is used for planning because it's the fastest and cheapest Claude model — the planning task only needs to produce a small JSON object, not a long answer.

Planner System Prompt

```

PLANNER_SYSTEM_PROMPT = """You are a research planning assistant.
Your job is to decompose a user's research question into exactly two targeted sub-queries:

1. A web search query – designed to find recent, factual, publicly-available information.

```

```
2. A RAG (Retrieval-Augmented Generation) query – designed to retrieve relevant passages from an internal document knowledge base.
```

Respond with ONLY a valid JSON object in this exact format (no markdown, no extra text):

```
{
  "web_search_query": "<query for live web search>",
  "rag_query": "<query for internal knowledge base>"
}
```

Why "no markdown, no extra text"? LLMs sometimes wrap JSON in code fences like ````json ... ````. The prompt explicitly forbids this, and the parser has a fallback that strips code fences anyway (defensive programming).

Why two separate query types? The web search query should be phrased for a search engine ("latest RAG 2024 benchmarks"), while the RAG query should be phrased for semantic similarity ("retrieval augmented generation techniques methods").

plan_node

```
async def plan_node(state: ResearchState) -> dict[str, Any]:
```

An `async` function — LangGraph calls `await plan_node(state)` when executing.

```
llm = ChatAnthropic(model=PLANNER_MODEL, temperature=0)
```

`temperature=0` — deterministic output. For a planning task you want the same decomposition every time, not creative variation.

```
messages = [
    SystemMessage(content=PLANNER_SYSTEM_PROMPT),
    HumanMessage(content=f"Research question: {state['query']}"),
]

response = await llm.ainvoke(messages)
raw_content: str = response.content.strip()
```

`llm.ainvoke()` is the async version of `llm.invoke()`. It sends the messages to Claude and awaits the response without blocking the event loop.

`response.content` is the raw text returned by Claude — should be a JSON string.


```

    try:
        plan = json.loads(raw_content)
    except json.JSONDecodeError as exc:
        if "`" in raw_content:
            inner = raw_content.split("`")[1]
            if inner.startswith("json"):
                inner = inner[4:]
                plan = json.loads(inner.strip())
            else:
                raise ValueError(...) from exc

```

Two-pass JSON parser:

- First try: `json.loads(raw_content)` — works if Claude followed instructions.
- Fallback: If `json.JSONDecodeError` is raised AND the response contains backticks, split on ``` and take the middle piece (the code block content). Strip the `json` language identifier if present, then parse again.
- If neither works, raise `ValueError` to let LangGraph mark the run as failed.

```

    return {
        "sub_tasks": [web_search_query, rag_query],
        "status": "planning",
    }

```

Nodes return a **partial state dict** — only the keys being updated. LangGraph merges this into the full `ResearchState`. Any key not in the return dict keeps its old value.

web_search_node

```

async def web_search_node(state: ResearchState) -> dict[str, Any]:
    query = state["sub_tasks"][0]
    result = await send_task(WEB_SEARCH_AGENT_URL, query)
    return {"web_results": result}

```

Simple: reads `sub_tasks[0]`, calls the A2A client, returns only `web_results`.

Does NOT return `status` — updating the same `status` field from both parallel nodes simultaneously would cause `INVALID_CONCURRENT_GRAPH_UPDATE`.

rag_search_node

```
async def rag_search_node(state: ResearchState) -> dict[str, Any]:
    query = state["sub_tasks"][1]
    result = await send_task(RAG_AGENT_URL, query)
    return {"rag_results": result}
```

Mirror of `web_search_node` — reads `sub_tasks[1]`, returns only `rag_results`.

synthesize_node

```
async def synthesize_node(state: ResearchState) -> dict[str, Any]:
    synthesis_input = json.dumps(
        {
            "query": state["query"],
            "web_results": state["web_results"],
            "rag_results": state["rag_results"],
        },
        ensure_ascii=False,
    )
```

Bundles the three key pieces into a JSON string for the Synthesis Agent.

`ensure_ascii=False` — allows non-ASCII characters (e.g., accented letters in source snippets) to pass through as-is rather than being escaped as `\uXXXX`.

```
    result = await send_task(SYNTHESIS_AGENT_URL, synthesis_input)
    return {
        "synthesis_input": synthesis_input,
        "final_report": result,
        "status": "synthesizing",
    }
```

Returns three fields: the input (for debugging), the final report, and a status label.

dispatch_parallel_searches — Fan-Out Edge

```
def dispatch_parallel_searches(state: ResearchState) -> list[Send]:
    return [
        Send("web_search_node", state),
        Send("rag_search_node", state),
    ]
```

This is the most important function in the file. It is an **edge function** — called by LangGraph after `plan_node` completes to determine what happens next.

Instead of returning a string (the name of the next node, normal edges), it returns a **list of Send objects**. Each `Send(node_name, state)` tells LangGraph: "schedule this node to run with this state snapshot."

What `Send` does internally:

1. LangGraph enqueues both nodes in its async event loop.
2. Both run concurrently via `asyncio` (not threads — truly async Python).
3. LangGraph collects both return dicts and merges them into the shared state.
4. Only after BOTH complete does it advance to the next edge.

This gives us the fan-out/fan-in pattern for free — no locks, no queues, no manual `asyncio.gather()`.

`build_graph` — Graph Assembly

```
def build_graph() -> Any:
    builder = StateGraph(ResearchState)
```

`StateGraph(ResearchState)` creates a new graph builder that validates every node's return dict against the `ResearchState` TypedDict.

```
builder.add_node("plan_node", plan_node)
builder.add_node("web_search_node", web_search_node)
builder.add_node("rag_search_node", rag_search_node)
builder.add_node("synthesize_node", synthesize_node)
```

Registers the four node functions with string labels. The string labels are what `Send("web_search_node", ...)` refers to.

```
builder.add_edge(START, "plan_node")
```

Normal edge: graph always starts at `plan_node`.

```
builder.add_conditional_edges(  
    "plan_node",  
    dispatch_parallel_searches,  
    ["web_search_node", "rag_search_node"],  
)
```

Conditional edge with `Send`:

- After `plan_node`, call `dispatch_parallel_searches(state)`.
- The third argument declares which nodes the function might route to — LangGraph uses this to validate the graph topology at compile time.
- The function returns `[Send(...), Send(...)]` — so both are scheduled in parallel.

```
builder.add_edge("web_search_node", "synthesize_node")  
builder.add_edge("rag_search_node", "synthesize_node")
```

Both parallel branches converge at `synthesize_node`. LangGraph waits for both before proceeding — this is the automatic fan-in.

```
builder.add_edge("synthesize_node", END)  
compiled = builder.compile()  
return compiled
```

`builder.compile()` validates the graph (checks for unreachable nodes, invalid edges) and returns a `CompiledGraph` object with an `ainvoke()` method.

Module-Level Execution

```
graph = build_graph()
```

The graph is built once when the module is first imported by `api/main.py`. All subsequent `/research` requests reuse the same compiled graph — no re-compilation overhead per request.

Why LangGraph?

Alternative	Problem
Manual <code>asyncio.gather()</code>	No state management, error handling must be hand-coded
LangChain Chains	Linear only, no branching or parallel fan-out
Celery/task queues	Heavyweight, separate broker needed, overkill for 4 nodes
LangGraph	State is a TypedDict, parallel branches are first-class, human-in-the-loop ready

Section 4

A2A Client

orchestrator/a2a_client.py — A2A HTTP Client

What this file does

Implements the **client side of the A2A (Agent-to-Agent) protocol** — the code that submits a task to a remote agent and polls for the result.

Every call from the orchestrator to a specialist agent goes through `send_task()`.

A2A Protocol in 30 Seconds



Why submit-then-poll instead of a blocking POST?

The agent's work can take 10–30 seconds (web searches, LLM calls). A blocking HTTP request that long would tie up connections and is fragile on network boundaries. The submit-then-poll pattern lets both sides stay responsive.

Full File: `orchestrator/a2a_client.py`

Imports

```
import asyncio
import logging
from uuid import uuid4
```



```
import httpx
```

- `asyncio` — for `asyncio.sleep()` between polls (non-blocking wait).
- `uuid4` — generates a random UUID for each task submission (unique identifier).
- `httpx` — async-native HTTP client. Unlike `requests`, it works inside an async event loop without blocking.

Constants

```
POLL_INTERVAL_SECONDS: float = 1.0
TIMEOUT_SECONDS: float = 60.0
HTTP_CONNECT_TIMEOUT: float = 5.0
HTTP_READ_TIMEOUT: float = 10.0
```

Two distinct timeout concepts:

- `HTTP_CONNECT_TIMEOUT` / `HTTP_READ_TIMEOUT` — per-request timeouts. Each individual HTTP call (POST or GET) must complete within these limits. 5s to establish a TCP connection, 10s to read the response body.
- `TIMEOUT_SECONDS` — overall task timeout. The total time we're willing to wait for the agent to finish. 60 seconds covers even slow Tavily + LLM calls.

If `HTTP_READ_TIMEOUT` fires, `httpx` raises `httpx.ReadTimeout`. If `TIMEOUT_SECONDS` is exceeded, `A2AError` is raised by the polling loop.

A2AError

```
class A2AError(Exception):
    """Raised when an A2A agent returns a failed status or cannot be reached."""
```

A custom exception class so callers can catch A2A-specific failures separately from other `Exception` types. The orchestrator lets this bubble up to `api/main.py`, which converts it to an HTTP 500 response.

`send_task()` — Step by Step

```
async def send_task(agent_url: str, message: str) -> str:
```

`async def` — this function is a coroutine. It can be awaited and yields control back to the event loop during `asyncio.sleep()` and `await client.post/get()`.

```
    task_id_request = str(uuid4())
    send_url = f"{agent_url}/tasks/send"
    poll_url_template = f"{agent_url}/tasks/{{task_id}}"
```

- `task_id_request` — a UUID we generate on the client side and send in the POST body. The server uses this as the request ID (idempotency), but generates its own internal `task_id` for the task store.
- The double braces `{{task_id}}` in the template string are escaped braces — they produce literal `{task_id}` in the string so we can `.format(task_id=...)` later.

```
    timeout_cfg = httpx.Timeout(
        connect=HTTP_CONNECT_TIMEOUT,
        read=HTTP_READ_TIMEOUT,
        write=HTTP_CONNECT_TIMEOUT,
        pool=HTTP_CONNECT_TIMEOUT,
    )
```

`httpx.Timeout` lets you set different timeouts for different phases of an HTTP request:

- `connect` — time to establish the TCP connection
- `read` — time to read the response body after the connection is open
- `write` — time to send the request body
- `pool` — time to acquire a connection from the connection pool

```
    async with httpx.AsyncClient(timeout=timeout_cfg) as client:
```

`async with` — creates the HTTP client and guarantees the underlying connection pool is closed when the block exits (even on exception). `httpx.AsyncClient` reuses TCP connections for multiple requests, which is efficient for the submit + multiple polls.

```
        send_response = await client.post(
            send_url,
            json={"id": task_id_request, "message": message},
        )
```

```
send_response.raise_for_status()
task_id: str = send_response.json()["task_id"]
```

- `await client.post(...)` — sends the HTTP POST without blocking the event loop.
- `raise_for_status()` — raises `httpx.HTTPStatusError` if the status code is 4xx or 5xx. This is the `httpx` equivalent of `requests.Response.raise_for_status()`.
- The server returns `{"task_id": "...", "status": "submitted"}`. We extract `task_id` for polling.

Polling Loop

```
elapsed: float = 0.0

while elapsed < TIMEOUT_SECONDS:
    await asyncio.sleep(POLL_INTERVAL_SECONDS)
    elapsed += POLL_INTERVAL_SECONDS
```

`await asyncio.sleep(1.0)` — pauses this coroutine for 1 second but does NOT block the event loop. Other coroutines (including the OTHER parallel agent call) continue running during this pause. This is the key difference from `time.sleep(1.0)`.

```
poll_response = await client.get(poll_url)
poll_response.raise_for_status()
payload = poll_response.json()
status: str = payload.get("status", "unknown")
```

Polls the task status endpoint. The server returns:

```
{"task_id": "...", "status": "working", "result": null}
```

or, when done:

```
{"task_id": "...", "status": "completed", "result": "...full text..."}
```

```
if status == "completed":
    result: str | None = payload.get("result")
```

```
        if result is None:
            raise A2AError(...)
        return result
```

On "completed", validate that `result` is not null (defensive: the server should always set result on completion, but we guard against bugs), then return the string.

```
        if status == "failed":
            error_detail = payload.get("result") or payload.get("error") or "no detail"
            raise A2AError(...)
```

If the agent marked the task as "failed", raise immediately — no point continuing to poll.

```
        raise A2AError(f"Timeout after {TIMEOUT_SECONDS}s ...")
```

If we exit the while loop naturally (elapsed $\geq$ 60s) without a terminal status, the task took too long.

Why httpx Instead of aiohttp or requests?

Library	Issue
<code>requests</code>	Synchronous — blocks the event loop. Would freeze all other async tasks.
<code>aiohttp</code>	Async but more complex API. httpx has a nearly identical API to <code>requests</code> .
<code>httpx</code>	Async + sync, familiar API, excellent timeout handling, actively maintained.

Timeline During Parallel Execution

When both `web_search_node` and `rag_search_node` call `send_task()` in parallel:

```
Time 0.0s web_search_node: POST /tasks/send to :8001
          rag_search_node: POST /tasks/send to :8002
Time 1.0s web_search_node: await sleep(1)
```

```
rag_search_node: await sleep(1)
(both sleeping simultaneously – event loop handles other work)
Time 1.0s web_search_node: GET /tasks/{id} → status "working"
rag_search_node: GET /tasks/{id} → status "working"
Time 2.0s web_search_node: GET /tasks/{id} → status "completed"
(returns immediately)
Time 3.0s rag_search_node: GET /tasks/{id} → status "completed"
(returns)
Time 3.0s LangGraph: both branches done, advance to synthesize_node
```

Total wall-clock time $\approx$ time of the SLOWER agent, not the SUM of both.

Section 5

Base A2A Server

agents/base_a2a_server.py — Shared A2A Server Factory

What this file does

Provides a **factory function** `create_a2a_app()` that builds a complete A2A-compliant FastAPI application. Every specialist agent (web search, RAG, synthesis) calls this function rather than writing the same boilerplate three times.

This is the server-side counterpart to `a2a_client.py`.

What the Factory Creates

For every agent, the returned FastAPI app exposes:

```
GET  /.well-known/agent.json  →  Agent Card (agent identity + capabilities)
POST /tasks/send              →  Submit a task, returns task_id immediately
GET  /tasks/{task_id}        →  Poll for task status + result
GET  /health                  →  Health check (for Docker, load balancers)
```

Data Models

```
class TaskRequest(BaseModel):
    id: str
    message: str
```

The body accepted by `POST /tasks/send`. `id` is the client-generated request UUID (for idempotency/tracing). `message` is the task payload — a plain-English query or a JSON string depending on the agent.

```
class TaskSubmittedResponse(BaseModel):
    task_id: str
    status: str = "submitted"
```

Returned immediately when a task is accepted. `task_id` is the server's internal ID used for polling. The client doesn't reuse its submitted `id` for polling.

```
class TaskStatusResponse(BaseModel):
    task_id: str
```



```
status: str          # "submitted" | "working" | "completed" | "failed"
result: str | None = None
```

Returned by the poll endpoint. `result` is `None` until the task completes or fails, then it holds the agent's output (or error message).

`create_a2a_app()` — Parameter by Parameter

```
def create_a2a_app(
    *,
    agent_name: str,
    agent_description: str,
    agent_version: str,
    capabilities: list[str],
    port: int,
    process_task: Callable[[str], Coroutine[Any, Any, str]],
) -> FastAPI:
```

- `*` — forces all parameters to be keyword-only. You can't call this as `create_a2a_app("Web Search Agent", ...)` — you must write `create_a2a_app(agent_name="Web Search Agent", ...)`. This prevents parameter order mistakes.
- `process_task: Callable[[str], Coroutine[Any, Any, str]]` — a type annotation saying "a callable that takes a string and returns a coroutine that resolves to a string." In plain English: an async function `async def process_task(msg: str) -> str`. This is the plug-in point where each agent injects its specific logic.

In-Memory Task Store

```
tasks: dict[str, dict[str, Any]] = {}
```

A plain Python dict defined inside the factory function closure. Each entry:

```
tasks["some-uuid"] = {"status": "submitted", "result": None}
```

Why a plain dict? For a portfolio agent running on a single process, this is sufficient. In production you'd use Redis or a database so tasks survive restarts and scale across multiple workers.

Closure note: `tasks` is captured by the three endpoint functions below. Each call to `create_a2a_app()` creates a new, independent `tasks` dict — so each agent has its own isolated task store.

Agent Card Endpoint

```
agent_card: dict[str, Any] = {
    "name": agent_name,
    "description": agent_description,
    "version": agent_version,
    "url": f"http://localhost:{port}",
    "capabilities": capabilities,
    "protocol": "a2a/1.0",
}

@app.get("/.well-known/agent.json", tags=["A2A"])
async def get_agent_card() -> dict[str, Any]:
    return agent_card
```

`/.well-known/` is a standardised URL path (RFC 8615) for machine-readable service metadata. `agent.json` is the A2A Agent Card — a self-description that lets orchestrators and registries discover what this agent does without reading source code.

Example response for the Web Search Agent:

```
{
  "name": "Web Search Agent",
  "description": "Searches the live web using the Tavily API...",
  "version": "1.0.0",
  "url": "http://localhost:8001",
  "capabilities": ["web_search", "real_time_information"],
  "protocol": "a2a/1.0"
}
```

POST /tasks/send

```
@app.post("/tasks/send", response_model=TaskSubmittedResponse)
async def send_task(request: TaskRequest) -> TaskSubmittedResponse:
    task_id = str(uuid.uuid4())
    tasks[task_id] = {"status": "submitted", "result": None}
```

A new UUID is generated server-side for every task. This is independent of the client's `request.id` — two different IDs on purpose: the client's ID is for its own tracking; the server's `task_id` is the key in the server's `tasks` dict.

```
    async def _run() -> None:
        tasks[task_id]["status"] = "working"
        try:
            result = await process_task(request.message)
            tasks[task_id]["status"] = "completed"
            tasks[task_id]["result"] = result
        except Exception as exc:
            tasks[task_id]["status"] = "failed"
            tasks[task_id]["result"] = f"Error: {exc}"

    asyncio.create_task(_run())
    return TaskSubmittedResponse(task_id=task_id, status="submitted")
```

`asyncio.create_task(_run())` — this is the core of the non-blocking design.

- `_run()` is a coroutine (inner async function).
- `asyncio.create_task()` schedules it to run on the event loop but does NOT await it.
- The outer `send_task` endpoint returns the `TaskSubmittedResponse` **immediately**, before `_run()` even starts.
- `_run()` executes in the background concurrently with future requests.

Why not await `_run()` directly? If we awaited it, the POST endpoint would block for 10–30 seconds before responding. The A2A protocol requires immediate acknowledgment so the client can poll separately.

noqa: BLE001 in the except clause — suppresses the "blind exception" linter warning. Catching bare `Exception` is intentional here: we don't want any agent failure to crash the server; instead we capture the error as a task result.

GET /tasks/{task_id}

```

@app.get("/tasks/{task_id}", response_model=TaskStatusResponse)
async def get_task(task_id: str) -> TaskStatusResponse:
    if task_id not in tasks:
        raise HTTPException(status_code=404, detail=f"Task '{task_id}' not found.")
    task = tasks[task_id]
    return TaskStatusResponse(
        task_id=task_id,
        status=task["status"],
        result=task["result"],
    )

```

Simple dict lookup. If the `task_id` doesn't exist (e.g., wrong ID, server restart, expired), returns HTTP 404. FastAPI automatically converts `HTTPException` to the appropriate JSON error response.

Task Lifecycle State Machine



The `status` field transitions through these four states. The A2A client polls until it sees `"completed"` or `"failed"`.

Factory Pattern — Why Not Subclassing?

An alternative design would be an `A2AAgent` base class with a `process_task` abstract method. The factory pattern was chosen instead because:

1. **Simplicity** — each agent is just a module with a `process_task` function and a call to `create_a2a_app()`. No class inheritance needed.
2. **Testability** — `create_a2a_app()` can be called in tests with a mock `process_task` function.
3. **No shared state between agents** — each call to `create_a2a_app()` creates a new independent `tasks` dict via closure.

Section 6

Web Search Agent

agents/web_search/agent.py — Web Search Agent

What this file does

Implements the **Web Search Agent** — a FastAPI A2A server on port 8001 that:

1. Receives a search query from the orchestrator
2. Calls the Tavily API to search the live web
3. Formats and returns the top 5 results (title, URL, snippet)

Imports and Path Setup

```
import logging
import os
import sys
import textwrap
from pathlib import Path

import uvicorn
from dotenv import load_dotenv
from tavily import TavilyClient

_PROJECT_ROOT = Path(__file__).resolve().parent.parent.parent
if str(_PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(_PROJECT_ROOT))
load_dotenv(_PROJECT_ROOT / ".env")

from agents.base_a2a_server import create_a2a_app
```

Path resolution:

```
agents/web_search/agent.py
.parent           → agents/web_search/
.parent.parent    → agents/
.parent.parent.parent → multi-agent-researcher/ ← _PROJECT_ROOT
```

`sys.path.insert(0, str(_PROJECT_ROOT))` adds the project root to Python's module search path, making `from agents.base_a2a_server import ...` work regardless of where you run the script from.

`load_dotenv(_PROJECT_ROOT / ".env")` reads the `.env` file to load `TAVILY_API_KEY` into the environment before any code tries to use it.

The check `if str(_PROJECT_ROOT) not in sys.path` prevents adding the same path twice if the module is imported multiple times.

Constants

```
PORT = 8001
MAX_RESULTS = 5
SNIPPET_MAX_CHARS = 300
```

- `MAX_RESULTS = 5` — limit Tavily results to 5. More results = larger payload sent to the synthesis agent, which uses more tokens.
- `SNIPPET_MAX_CHARS = 300` — each snippet is truncated to 300 characters. Without this, a single web result could be several paragraphs, bloating the synthesis prompt.

`_get_tavily_client()`

```
def _get_tavily_client() -> TavilyClient:
    api_key = os.getenv("TAVILY_API_KEY")
    if not api_key:
        raise RuntimeError(
            "TAVILY_API_KEY is not set. ..."
        )
    return TavilyClient(api_key=api_key)
```

The client is created on every call to `process_task()` rather than at module load time. This is intentional: if `TAVILY_API_KEY` is missing, the error surfaces at request time with a clear message, rather than crashing the entire server at startup.

This also makes testing easier — you can patch `os.getenv` to inject a test key.

`process_task()` — Core Logic

```
async def process_task(message: str) -> str:
    logger.info("Web search query: %r", message)
```

```
client = _get_tavily_client()
```

`%r` in the log format uses `repr()` on the value — it quotes strings, making it clear in logs that the message is a string and showing any special characters.

```
response = client.search(  
    query=message,  
    search_depth="basic",  
    max_results=MAX_RESULTS,  
    include_answer=False,  
    include_raw_content=False,  
)
```

Tavily `search()` parameters:

- `search_depth="basic"` — faster and cheaper than `"advanced"`. Advanced gives richer, full-page content; basic gives snippets from search result pages.
- `include_answer=False` — Tavily can optionally synthesize its own answer from results. We skip this because our Synthesis Agent will do the synthesis.
- `include_raw_content=False` — raw HTML/full text of pages. Too large; we only need the snippet Tavily already extracts.

The `TavilyClient.search()` method is **synchronous** (it blocks). The base A2A server calls `process_task` inside `asyncio.create_task(_run())`, which runs it in an async context. For a portfolio project this is fine since the task is already in a background coroutine. For high throughput you'd wrap it with `asyncio.to_thread()`.

```
results = response.get("results", [])  
  
if not results:  
    return f"No web results found for query: {message!r}"
```

Defensive: Tavily might return empty results for very obscure queries. Return a useful message rather than an empty string so the synthesis agent has something to work with.

```
lines: list[str] = [f"Web search results for: {message!r}\n"]  
  
for i, item in enumerate(results[:MAX_RESULTS], start=1):  
    title = item.get("title", "No title").strip()  
    url = item.get("url", "").strip()  
    content = item.get("content", "").strip()
```

```

    snippet = textwrap.shorten(content, width=SNIPPET_MAX_CHARS, placeholder="...")

    lines.append(
        f"{i}. {title}\n"
        f"    URL: {url}\n"
        f"    {snippet}"
    )

    return "\n\n".join(lines)

```

textwrap.shorten(content, width=300, placeholder="..."):

- Truncates `content` to at most 300 characters at a word boundary.
- Appends "..." if truncation occurred.
- Unlike `content[:300]`, `shorten` never cuts a word in half.

Output format example:

```

Web search results for: 'latest RAG systems 2024'

1. Advances in Retrieval-Augmented Generation
   URL: https://example.com/rag-advances
   RAG systems have evolved significantly in 2024, with new approaches...

2. Graph RAG: Knowledge Graph + Vector Retrieval
   URL: https://example.com/graph-rag
   GraphRAG combines entity extraction with vector search to improve...

```

This format is human-readable and works well as context for the Synthesis Agent's prompt.

App Registration

```

app = create_a2a_app(
    agent_name="Web Search Agent",
    agent_description=(
        "Searches the live web using the Tavily API and returns the top 5 results "
        "(title, URL, snippet) for any research query."
    ),
)

```

```
    agent_version="1.0.0",
    capabilities=["web_search", "real_time_information"],
    port=PORT,
    process_task=process_task,
)
```

One line for the entire A2A server setup. The factory creates the FastAPI app with all four A2A endpoints wired up. `process_task` is the single injection point for this agent's specific logic.

Entry Point

```
if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=PORT)
```

`host="0.0.0.0"` — listens on all network interfaces. Required in Docker (if you used `127.0.0.1`, the container would only be accessible to itself).

`if __name__ == "__main__"` — this block only runs when the file is executed directly (`python agents/web_search/agent.py`). When imported as a module (e.g., in tests), uvicorn is not started.

Tavily API

Tavily is a search API built specifically for LLM applications. Unlike raw Google/Bing APIs:

- Results come pre-cleaned (no HTML to parse)
- Snippets are formatted for use as LLM context
- Free tier: 1000 API calls/month
- `search_depth="basic"` is ~2× faster than `"advanced"` and uses half the quota

Sign up at <https://app.tavily.com> to get `TAVILY_API_KEY`.

Section 7

Rag Agent

agents/rag_agent/agent.py — RAG Agent

What this file does

Implements the **RAG Agent** — a FastAPI A2A server on port 8002 that queries the `rag-doc-qa` knowledge base and returns a cited answer.

This agent is a **thin wrapper**: it doesn't contain any retrieval or generation logic. Instead, it reuses the production `answer_question()` function from the sibling `rag-doc-qa` project by injecting that project's root onto `sys.path`.

The Reuse Strategy — `sys.path` Surgery

```
_THIS_FILE = Path(__file__).resolve()
_AGENTS_DIR = _THIS_FILE.parent.parent # ../multi-agent-researcher/agents
_PROJECT_ROOT = _AGENTS_DIR.parent # ../multi-agent-researcher
_RAG_PROJECT_ROOT = _PROJECT_ROOT.parent / "rag-doc-qa"
```

Path tree:

```
portfolio_projects/
├── multi-agent-researcher/
│   ├── agents/
│   │   └── rag_agent/
│   │       └── agent.py ← _THIS_FILE
│   └── rag-doc-qa/ ← _RAG_PROJECT_ROOT
│       ├── app/
│       └── services/
│           └── qa_chain.py
```

```
if not _RAG_PROJECT_ROOT.exists():
    raise RuntimeError(
        f"rag-doc-qa project not found at expected path: {_RAG_PROJECT_ROOT}\n"
        "Make sure both projects are siblings inside the same portfolio_projects/ folder."
    )
```

Fail fast with a clear error message if the folder structure doesn't match. This is much better than getting an obscure `ModuleNotFoundError` 10 lines later.

```
if str(_RAG_PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(_RAG_PROJECT_ROOT))
if str(_PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(_PROJECT_ROOT))
```

Two path injections:

1. `_RAG_PROJECT_ROOT` — so `from app.services.qa_chain import answer_question` works (rag-doc-qa uses `app/` as the top-level package).
2. `_PROJECT_ROOT` — so `from agents.base_a2a_server import create_a2a_app` works.

Why Reuse Instead of Duplicate?

The RAG pipeline (ChromaDB + BM25 + Claude) involves:

- ~500 lines across 5 service files
- Embedded models (sentence-transformers, 90MB)
- A persisted ChromaDB vector store

Duplicating this would mean:

- Maintaining two copies of the same code
- Keeping two separate ChromaDB databases in sync when documents are uploaded
- Double the disk space for models

By injecting `sys.path`, the RAG Agent uses the **exact same database and models** as rag-doc-qa. Any document uploaded through the Orchestrator's `/upload` endpoint immediately becomes searchable by the RAG Agent.

`_format_sources()`

```
def _format_sources(sources: list[dict]) -> str:
    if not sources:
        return "No sources retrieved."
    lines: list[str] = ["Sources:"]
    for i, src in enumerate(sources[:MAX_SOURCES_DISPLAYED], start=1):
        filename = src.get("filename", "unknown")
        page = src.get("page")
        snippet = src.get("snippet", "")
        page_label = f", page {page}" if page else ""
        lines.append(f"  {i}. {filename}{page_label} – {snippet}")
```



```
return "\n".join(lines)
```

Takes the `sources` list from `answer_question()` (list of dicts with filename, page, snippet keys) and formats it as readable text.

`page_label = f", page {page}" if page else ""` — conditional expression. If the page number is available, include it; for TXT files without pages, omit it gracefully.

`sources[:MAX_SOURCES_DISPLAYED]` — caps at 5 sources. More than 5 citations in the synthesis prompt adds noise without improving quality.

`process_task()` — Core Logic

```
async def process_task(message: str) -> str:
    logger.info("RAG query: %r", message)

    import asyncio
    result = await asyncio.to_thread(answer_question, message, [])
```

`asyncio.to_thread(answer_question, message, [])` — this is critical.

`answer_question()` from `rag-doc-qa` is a **synchronous** function that:

1. Calls `hybrid_search()` — synchronous ChromaDB + BM25 query
2. Calls Claude via the Anthropic SDK — synchronous HTTP call

Both operations block the current thread. If called directly inside `async def process_task()`, they would block the entire asyncio event loop, preventing any other task (including the Web Search Agent's polling loop) from progressing.

`asyncio.to_thread()` solves this by:

1. Picking a thread from Python's `ThreadPoolExecutor`
2. Running `answer_question(message, [])` in that thread
3. Awaiting the thread's completion without blocking the event loop

The second argument `[]` is the chat history — empty because each RAG query is standalone (no conversation memory in this pipeline).

```
answer = result.get("answer", "No answer generated.")
sources = result.get("sources", [])
```

```
sources_text = _format_sources(sources)

return f"Answer: {answer}\n\n{sources_text}"
```

`answer_question()` returns `{"answer": str, "sources": list[dict]}`. We extract both fields and format them into a single string for the Synthesis Agent.

Output format example:

```
Answer: RAG systems in 2024 have focused on hybrid retrieval combining dense
vector search with sparse BM25 for improved precision. Key advances
include GraphRAG and ColBERT-based late interaction retrieval.

Sources:
1. rag_overview.pdf, page 3 – Dense retrieval methods have evolved to incorporate...
2. rag_benchmarks.pdf, page 7 – BM25 combined with semantic search via RRF shows...
```

App Registration

```
app = create_a2a_app(
    agent_name="RAG Agent",
    agent_description=(
        "Queries an uploaded document knowledge base using hybrid semantic + BM25 "
        "retrieval and generates citation-backed answers via Claude."
    ),
    agent_version="1.0.0",
    capabilities=["document_qa", "hybrid_retrieval", "citation_generation"],
    port=PORT,
    process_task=process_task,
)
```

Same pattern as the Web Search Agent — one call to the factory, everything else is handled by `base_a2a_server.py`.

Dependency Chain

```
RAG Agent: process_task(message)
```

```
    """ asyncio.to_thread(answer_question, message, [])
```

```
        """ rag-doc-qa/app/services/qa_chain.py::answer_question()
```

```
            """ retriever.hybrid_search(message, k=5)
```

```
            """ ChromaDB semantic search (cosine similarity)
```

```
            """ BM25 keyword search + RRF fusion
```

```
            """ Claude (via Anthropic SDK) → answer with citations
```

Everything below `asyncio.to_thread()` runs in a background thread and is borrowed entirely from the rag-doc-qa project. The RAG Agent adds only:

- A2A protocol wrapping (via `base_a2a_server`)
- `_format_sources()` for readable text output
- `asyncio.to_thread()` to keep the event loop unblocked

Section 8

Synthesis Agent

agents/synthesis/agent.py — Synthesis Agent

What this file does

Implements the **Synthesis Agent** — a FastAPI A2A server on port 8003 that receives the original query plus both sets of research results, then uses Claude to produce a polished markdown research report.

This is the final step in the pipeline before the report is returned to the user.

Input Format

The orchestrator sends a JSON string (not plain text) to this agent:

```
{
  "query": "What are the latest advances in RAG systems?",
  "web_results": "Web search results for: ...\n\n1. Title\n  URL: ...",
  "rag_results": "Answer: ...\n\nSources:\n  1. doc.pdf, page 3 - ..."
}
```

The agent must parse this JSON, format a prompt, call Claude, and return the report.

Constants

```
PORT = 8003
CLAUDE_MODEL = "claude-sonnet-4-6"
MAX_TOKENS = 2048
```

`claude-sonnet-4-6` is used here (the most capable model) because synthesis requires:

- Understanding two sets of results that may use different vocabulary
- Identifying contradictions between web and knowledge base sources
- Producing structured markdown with specific section headers
- Writing 300–600 words concisely

Claude Haiku would work but produces lower-quality reports for this task.

`MAX_TOKENS = 2048` — generous enough for a 600-word report with citations, but bounded to prevent runaway costs.

System Prompt

```
SYNTHESIS_SYSTEM_PROMPT = """\nYou are a senior research analyst. Your job is to synthesise information from\ntwo sources – live web search results and a curated document knowledge base –\ninto a structured, balanced research report.\n\nRules:\n- Write exclusively in Markdown.\n- Be factual and precise; do not hallucinate facts not present in the sources.\n- If the two sources contradict each other, note the discrepancy explicitly.\n- Keep the report concise but complete (300–600 words is ideal).\n- Use the section headers exactly as specified.\n\n"""
```

Key rules:

- **"Write exclusively in Markdown"** — the Streamlit UI renders this with `st.markdown()`, so markdown headers, bullets, and bold text display properly.
- **"Do not hallucinate"** — grounding instruction. Prevents Claude from adding facts from its training data that aren't in the provided sources.
- **"If the two sources contradict each other, note the discrepancy"** — this is a distinctive feature for a research tool. Web results reflect current public information; the knowledge base may contain older or organization-specific data.
- **"300–600 words"** — avoids both too-short (useless) and too-long (overwhelming) reports.

User Prompt Template

```
SYNTHESIS_USER_TEMPLATE = """\nResearch query: {query}\n\n--- WEB SEARCH RESULTS ---\n{web_results}\n\n--- KNOWLEDGE BASE RESULTS ---\n{rag_results}\n\n---\n\nPlease produce a structured research report with the following sections:
```

```
## Summary
```

```
A 2-3 sentence overview of the topic.
```

```
## Key Findings (from web)
```

```
Bullet points highlighting the most relevant facts from the web results.
```

```
## From Knowledge Base
```

```
What the uploaded document knowledge base adds or confirms.
```

```
## Conclusion
```

```
A brief synthesis and any caveats or areas of uncertainty.
```

```
"""
```

Why include the section headers in the prompt? LLMs follow instructions more reliably when you show the exact format rather than describe it. The system prompt says "use the section headers exactly as specified" and this template shows them — double reinforcement.

`{query}`, `{web_results}`, `{rag_results}` — Python `.format()` placeholders.

Filled in by `_parse_input()` + `SYNTHESIS_USER_TEMPLATE.format(...)`.

`_get_anthropic_client()`

```
def _get_anthropic_client() -> anthropic.Anthropic:
    api_key = os.getenv("ANTHROPIC_API_KEY")
    if not api_key:
        raise RuntimeError("ANTHROPIC_API_KEY is not set. ...")
    return anthropic.Anthropic(api_key=api_key)
```

Uses the **raw Anthropic Python SDK** directly (not through LangChain). Either works; this shows you can call the Anthropic API directly without any framework wrapper.

The client is created per-task rather than as a module-level singleton. This is safe because `anthropic.Anthropic` objects are lightweight (they hold the API key and configuration but don't maintain persistent connections).

`_parse_input()`

```
def _parse_input(message: str) -> tuple[str, str, str]:
    try:
        data = json.loads(message)
    except json.JSONDecodeError as exc:
        raise ValueError(
            f"Synthesis agent expected a JSON string but received invalid JSON: {exc}\n"
            f"Raw message: {message[:200]}"
        ) from exc
```

`json.loads(message)` — parses the JSON string sent by `synthesize_node` in `graph.py`.

`raise ValueError(...) from exc` — the `from exc` clause chains the original `JSONDecodeError` as the cause of the new `ValueError`. This preserves the full traceback for debugging while providing a cleaner error message.

```
if not web_results:
    web_results = "No web search results were available for this query."
if not rag_results:
    rag_results = "No knowledge base results were available for this query."
```

Fallback values ensure the prompt is always well-formed even if one agent returned empty results. The synthesis agent can still produce a partial report.

`process_task()` — Core Logic

```
async def process_task(message: str) -> str:
    query, web_results, rag_results = _parse_input(message)

    user_prompt = SYNTHESIS_USER_TEMPLATE.format(
        query=query,
        web_results=web_results,
        rag_results=rag_results,
    )

    client = _get_anthropic_client()
```

`SYNTHESIS_USER_TEMPLATE.format(...)` — standard Python string formatting. Replaces `{query}`, `{web_results}`, `{rag_results}` with their actual values.

```
def _call_claude() -> str:
    response = client.messages.create(
        model=CLAUDE_MODEL,
        max_tokens=MAX_TOKENS,
        system=SYNTHESIS_SYSTEM_PROMPT,
        messages=[{"role": "user", "content": user_prompt}],
    )
    return response.content[0].text

report: str = await asyncio.to_thread(_call_claude)
```

Why wrap in `asyncio.to_thread()`?

`client.messages.create()` is the Anthropic SDK's **synchronous** API call. It makes an HTTP request and blocks until Claude responds — which can take 5–15 seconds for a long synthesis.

If called directly inside `async def process_task()`, it would block the asyncio event loop for that duration. During this time:

- No other coroutines can run on that thread
- The HTTP response to the A2A POST `/tasks/send` would be delayed
- The orchestrator's polling requests would time out

`asyncio.to_thread(_call_claude)` runs `_call_claude` in a thread pool worker. The event loop remains unblocked — it can handle other requests and coroutines while Claude generates the report.

```
response.content[0].text
```

`client.messages.create()` returns a `Message` object. `content` is a list of content blocks (usually just one). Each block has a `type` ("text" or "tool_use") and `text`. We take `content[0].text` — the first (and only) text block.

Anthropic SDK vs LangChain

Raw Anthropic SDK

```
import anthropic
```

LangChain ChatAnthropic

```
from langchain_anthropic import
ChatAnthropic
```

Call	<code>client.messages.create(model=..., messages=[...])</code>	<code>llm.invoke([HumanMessage(...)])</code>
Retrieve	<code>response.content[0].text</code>	<code>response.content</code> (string directly)
Async	<code>client.messages.create()</code> (sync) / <code>AsyncAnthropic</code>	<code>llm.ainvoke()</code> (async)
Used in	Synthesis Agent	<code>plan_node</code> in <code>graph.py</code>

Both work fine. The synthesis agent uses the raw SDK to show that LangChain is optional.

Output Example

```
## Summary
RAG (Retrieval-Augmented Generation) systems have advanced significantly in 2024,
with new approaches combining sparse and dense retrieval, graph-based knowledge
integration, and improved citation generation.

## Key Findings (from web)
- GraphRAG uses entity extraction and knowledge graphs for multi-hop reasoning
- ColBERT-based late interaction improves retrieval precision by 15% on BEIR benchmarks
- Hybrid BM25 + semantic search is now the production standard at most AI companies

## From Knowledge Base
The uploaded research papers confirm hybrid retrieval effectiveness, citing a 23%
improvement in answer accuracy vs. pure semantic search on domain-specific corpora.
No contradictions found between web and knowledge base sources.

## Conclusion
RAG systems are maturing rapidly. The main uncertainty is around scaling graph-based
approaches to very large document collections efficiently.
```

Section 9

Mcp Server

mcp_server/server.py — MCP Server

What this file does

Implements a **Model Context Protocol (MCP) server** that wraps the rag-doc-qa knowledge base and exposes it as callable tools to any MCP-compatible client (Claude Desktop, LangGraph tool node, etc.).

Important: The MCP server is a standalone tool — it is NOT used at runtime by the main research pipeline. The RAG Agent imports rag-doc-qa directly. The MCP server exists to let you connect Claude Desktop to the same knowledge base for interactive Q&A without running the full pipeline.

What is MCP?

Model Context Protocol is an open standard (from Anthropic, 2024) that defines how LLM clients connect to external tools and data sources. It's like a USB standard for AI tools — any MCP client can connect to any MCP server.

```
MCP Client (e.g. Claude Desktop)      MCP Server (this file)
┌───┐                                ┌───┐
│   │                                │   │
│   │                                │   │
│   │ list_tools() ──────────────────┤   │
│   │ ──────────────────────────── [search_documents, ...] ───────────┤   │
│   │                                │   │
│   │                                │   │
│   │ call_tool("search_documents", {...}) ───────────┤   │
│   │ ──────────────────────────── TextContent(text) ───────────┤   │
│   │                                │   │
```

Transport: stdio (standard input/output). The client spawns the server as a subprocess and communicates by writing JSON messages to its stdin and reading JSON responses from its stdout. This is why logging goes to `stderr` — stdout is reserved for the MCP protocol.

Path Bootstrap

```
RAG_PROJECT_ROOT = Path(__file__).parent.parent.parent / "rag-doc-qa"
sys.path.insert(0, str(RAG_PROJECT_ROOT))
```

Same pattern as the RAG Agent — injects `rag-doc-qa/` onto `sys.path` so the server can import `app.services.*` from that project.

MCP SDK Imports

```
from mcp.server import NotificationOptions, Server
from mcp.server.models import InitializationOptions
from mcp.server.stdio import stdio_server
from mcp.types import (
    CallToolRequest,
    CallToolResult,
    ListToolsRequest,
    ListToolsResult,
    TextContent,
    Tool,
)
```

- `Server` — the main MCP server class. Handles protocol-level message parsing.
- `stdio_server` — async context manager that sets up the stdin/stdout transport.
- `Tool` — describes a single tool (name, description, JSON schema for inputs).
- `TextContent` — the return type for tool results. MCP supports multiple content types (text, image, resource); we only use `TextContent`.
- `InitializationOptions` — configuration sent to the client during the MCP handshake.

Lazy Loading Pattern

```
_retriever = None
_qa_chain = None
_vector_store = None

def _get_retriever():
    global _retriever
    if _retriever is None:
        logger.info("Lazy-loading rag-doc-qa retriever ...")
        from app.services import retriever
        _retriever = retriever
    return _retriever
```

Three module-level `None` variables serve as lazy singletons.

Why lazy loading?

1. **Fast startup** — the MCP server starts in under 1 second. If we imported sentence-transformers at module load time, startup would take 10+ seconds.
2. **Conditional loading** — if only `list_documents` is called, the retriever and `qa_chain` are never loaded at all.
3. **Error isolation** — if ChromaDB data is missing, the error surfaces at tool-call time with a clear message, not at server startup.

`global _retriever` — declares that the assignment inside the function modifies the module-level variable, not a new local variable.

Tool Definitions

```
TOOLS: list[Tool] = [
    Tool(
        name="search_documents",
        description="...",
        inputSchema={
            "type": "object",
            "properties": {
                "query": {"type": "string", "description": "..."},
                "k": {"type": "integer", "default": 5, "minimum": 1, "maximum": 20},
            },
            "required": ["query"],
        },
    ),
    ...
]
```

`inputSchema` is a **JSON Schema** object. MCP clients use this to:

- Validate tool call arguments before sending
- Generate UI form fields for tool inputs
- Let Claude understand what arguments to provide

`"required": ["query"]` — `k` is optional (has a `default`), `query` is mandatory.

Tool Handlers


```
@server.list_tools()
async def handle_list_tools() -> list[Tool]:
    return TOOLS
```

`@server.list_tools()` is a decorator that registers this function as the handler for the MCP `ListTools` request. When a client asks "what tools do you have?", this returns the `TOOLS` list.

```
@server.call_tool()
async def handle_call_tool(name: str, arguments: dict) -> list[TextContent]:
```

`@server.call_tool()` registers this as the handler for all tool calls. The `name` parameter is the tool name; `arguments` is the parsed JSON input.

search_documents Handler

```
if name == "search_documents":
    query = arguments.get("query", "")
    k = int(arguments.get("k", 5))

    retriever = _get_retriever()
    results = await asyncio.to_thread(retriever.hybrid_search, query, k)
```

`asyncio.to_thread()` — `hybrid_search()` is synchronous (ChromaDB + BM25). Wrapping it prevents blocking the MCP server's async event loop.

```
documents = results.get("documents", [[]])[0]
metadatas = results.get("metadatas", [[]])[0]
ids = results.get("ids", [[]])[0]
```

ChromaDB returns results in a nested-list format: `{"documents": [["chunk1", "chunk2"]]}` (outer list is per-batch, inner list is per-result). We take `[0]` to get the flat list.

```
chunks = []
for doc, meta, chunk_id in zip(documents, metadatas, ids):
    chunks.append({
        "chunk_id": chunk_id,
        "text": doc,
```

```

        "source_filename": meta.get("source_filename", "unknown"),
        ...
    })
]

payload = {"query": query, "k": k, "num_results": len(chunks), "chunks": chunks}
return [TextContent(type="text", text=json.dumps(payload, ensure_ascii=False, indent=2))]

```

All tool results are returned as `list[TextContent]`. MCP requires this wrapping even for JSON — the content type is "text" and the `text` field holds the JSON string.

`ensure_ascii=False` — allows non-ASCII (accented characters, etc.) to pass through rather than being escaped.

`answer_with_rag` Handler

```

elif name == "answer_with_rag":
    qa = _get_qa_chain()
    result = await asyncio.to_thread(qa.answer_question, question)
    return [TextContent(type="text", text=json.dumps(result, ...))]

```

Calls `answer_question()` from `rag-doc-qa` — same function used by the RAG Agent. Returns `{"answer": str, "sources": list[dict]}` as a JSON string.

`list_documents` Handler

```

elif name == "list_documents":
    vs = _get_vector_store()
    docs = await asyncio.to_thread(vs.get_all_documents_info)
    payload = {"num_documents": len(docs), "documents": docs}
    return [TextContent(type="text", text=json.dumps(payload, ...))]

```

Lists all documents currently in ChromaDB. Useful for knowing what's indexed before deciding which search to run.

Entry Point

```

async def main() -> None:
    async with stdio_server() as (read_stream, write_stream):
        init_options = InitializationOptions(
            server_name="rag-mcp-server",
            server_version="1.0.0",
            capabilities=server.get_capabilities(
                notification_options=NotificationOptions(),
                experimental_capabilities={},
            ),
        )
        await server.run(read_stream, write_stream, init_options)

if __name__ == "__main__":
    asyncio.run(main())

```

`stdio_server()` — async context manager that:

1. Sets up stdin as the read stream
2. Sets up stdout as the write stream
3. Handles buffering and framing of MCP JSON messages

`server.run(read_stream, write_stream, init_options)` — starts the MCP protocol loop, reading requests and dispatching them to the registered handlers indefinitely.

`asyncio.run(main())` — creates the event loop and runs `main()` synchronously from the `__main__` entry point.

Connecting to Claude Desktop

Add to `%APPDATA%\Claude\claude_desktop_config.json`:

```

{
  "mcpServers": {
    "rag-knowledge-base": {
      "command": "python",
      "args": ["C:/Users/bvnik/portfolio_projects/multi-agent-researcher/mcp_server/server.py"]
    }
  }
}

```

Claude Desktop will spawn the MCP server as a subprocess on startup. You can then ask Claude "search my documents for X" and it will call the `search_documents` tool.

Why stdio Transport?

MCP supports three transports: stdio, HTTP+SSE, and WebSocket. stdio is used here because:

- Zero configuration — no ports, no URLs, no network setup
- Built-in security — only the parent process can communicate
- Claude Desktop's native integration uses stdio for local servers
- No separate server process to manage — the client starts and stops it automatically

Section 10

Api Main

api/main.py — FastAPI Orchestrator Entry Point

What this file does

The **HTTP entry point** for the entire multi-agent system. Runs on port 8000 and exposes three endpoints:

- `POST /research` — accepts a query, runs the full LangGraph pipeline, returns the report
- `POST /upload` — uploads a document to the knowledge base
- `GET /documents` — lists all indexed documents

The Streamlit UI talks exclusively to this service.

Startup: Environment and Path Setup

```
from pathlib import Path
from dotenv import load_dotenv

_PROJECT_ROOT = Path(__file__).resolve().parent.parent
load_dotenv(_PROJECT_ROOT / ".env")
```

`Path(__file__).resolve().parent.parent` — resolves to `multi-agent-researcher/` (two levels up from `api/main.py`).

`load_dotenv(...)` is called **before** any other import that might use environment variables. This is critical — if you import `orchestrator.graph` before loading `.env`, the `ChatAnthropic` constructor won't find `ANTHROPIC_API_KEY` and will raise an authentication error.

```
_RAG_ROOT = _PROJECT_ROOT.parent / "rag-doc-qa"
if str(_RAG_ROOT) not in sys.path:
    sys.path.insert(0, str(_RAG_ROOT))
```

Injects the `rag-doc-qa` project root so the `/upload` and `/documents` endpoints can import `process_upload` and `get_all_documents_info` from `rag-doc-qa`.

FastAPI Application

```

app = FastAPI(
    title="Multi-Agent Research Orchestrator",
    description="...",
    version="1.0.0",
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

```

`CORSMiddleware` — the Streamlit UI runs on port 8501 and this API is on port 8000.

A browser would block this cross-origin request without CORS headers. `allow_origins=["*"]` allows any origin during development. In production you'd restrict to your domain.

`allow_credentials=True` — allows cookies/auth headers in cross-origin requests.

Request/Response Schemas

```

class ResearchRequest(BaseModel):
    query: str = Field(..., min_length=3, max_length=2000, ...)

```

- `...` (Ellipsis) as the first argument — makes `query` required (no default).
- `min_length=3` — rejects one- or two-character inputs that can't be real questions.
- `max_length=2000` — prevents massive prompts from being passed in.

FastAPI validates incoming request bodies against this model automatically. If `query` is missing or fails validation, FastAPI returns HTTP 422 before your handler runs.

```

class ResearchResponse(BaseModel):
    query: str
    final_report: str
    status: str
    web_results: str
    rag_results: str

```


Defines the exact shape of the JSON response. FastAPI serializes the returned `ResearchResponse` object to JSON and validates it before sending.

POST /research — Main Endpoint

```
@app.post("/research", response_model=ResearchResponse)
async def run_research(request: ResearchRequest) -> ResearchResponse:
```

`async def` — FastAPI runs async endpoints on the event loop, allowing other requests to be handled concurrently while `graph.ainvoke()` is running.

```
    initial_state: ResearchState = {
        "query": request.query,
        "sub_tasks": [],
        "web_results": "",
        "rag_results": "",
        "synthesis_input": "",
        "final_report": "",
        "status": "starting",
        "error": None,
    }
```

Every field in `ResearchState` must be explicitly initialized — LangGraph validates the TypedDict from the first node. If any required field is missing, LangGraph raises a validation error before executing any node.

```
    try:
        result: ResearchState = await graph.ainvoke(initial_state)
    except Exception as exc:
        logger.error(...)
        raise HTTPException(status_code=500, detail=f"Research pipeline failed: {exc}")
```

`await graph.ainvoke(initial_state)` — runs the entire LangGraph pipeline asynchronously and returns the final `ResearchState` dict when all nodes have completed.

If any node raises an unhandled exception (network error, Claude API failure, etc.), it propagates here as a Python exception. We catch it and convert to HTTP 500 with a human-readable detail string.

```

    if not result.get("final_report"):
        raise HTTPException(
            status_code=500,
            detail="Research pipeline completed but produced no final report.",
        )

```

Double-check: the graph could theoretically "succeed" (no exceptions) but still produce an empty report if the synthesis agent returned an empty string. Guard against this edge case explicitly.

```

    return ResearchResponse(
        query=result["query"],
        final_report=result["final_report"],
        status=result["status"],
        web_results=result["web_results"],
        rag_results=result["rag_results"],
    )

```

Maps fields from `ResearchState` (the graph's TypedDict) to `ResearchResponse` (the API's Pydantic model). FastAPI serializes this to JSON automatically.

POST /upload — Document Upload

```

@app.post("/upload", tags=["knowledge-base"])
async def upload_document(file: UploadFile = File(...)):

```

`UploadFile` — FastAPI's type for multipart file uploads. Provides:

- `file.filename` — original filename
- `file.read()` — async read of file bytes
- `file.type` — MIME type (e.g., "application/pdf")

`File(...)` — the `...` ellipsis makes the file required.

```

    from app.services.document_processor import process_upload

    contents = await file.read()
    result = await asyncio.to_thread(process_upload, file.filename, contents)

```

Why import inside the function?

The `app.services.document_processor` module imports sentence-transformers and other heavy libraries at import time. Importing it at the module level would slow down API startup even for requests that never use the upload feature.

`asyncio.to_thread(process_upload, ...)` — `process_upload` is synchronous (it writes files, loads models, builds ChromaDB indexes). Running it in a thread keeps the event loop responsive.

```
    return {
        "message": f"Document '{file.filename}' uploaded successfully.",
        "doc_id": result["doc_id"],
        "num_chunks": result["num_chunks"],
        "num_pages": result.get("num_pages"),
    }
```

`result.get("num_pages")` — uses `.get()` (returns `None` if key missing) because TXT files don't have pages; only PDFs provide a page count.

GET /documents — List Knowledge Base

```
@app.get("/documents", tags=["knowledge-base"])
async def list_documents():
    from app.services.vector_store import get_all_documents_info
    docs = await asyncio.to_thread(get_all_documents_info)
    return {"documents": docs}
```

Same pattern: lazy import + `asyncio.to_thread`. Called by the Streamlit UI to show the "X document(s) in knowledge base" list below the upload form.

GET /health — Liveness Probe

```
@app.get("/health", tags=["ops"])
async def health() -> dict[str, str]:
    return {"status": "ok"}
```

Returns immediately without checking external dependencies (ChromaDB, agent URLs). Used by Docker health checks to know when the container is ready to serve traffic.

Module-Level Graph Import

```
from orchestrator.graph import graph
```

This import happens when the module is first loaded by uvicorn. `graph.py`'s module-level `graph = build_graph()` runs at this point — the LangGraph state machine is compiled once at startup, not once per request.

Startup sequence:

1. uvicorn loads `api.main`
2. `load_dotenv()` runs — `.env` is loaded
3. `from orchestrator.graph import graph` triggers graph compilation
4. FastAPI app is ready — all `/research` requests share the same compiled graph

Running in Development

```
uvicorn api.main:app --reload
```

`--reload` — uvicorn watches for file changes and restarts automatically. Useful during development. Remove in production.

The `api.main:app` syntax means: module `api.main`, attribute `app` (the FastAPI instance).

Section 11

Ui App

ui/app.py — Streamlit Frontend

What this file does

Provides the **user interface** for the research assistant. Runs on port 8501 and gives users:

1. A document upload form to add files to the knowledge base
2. A research query input box
3. A rendered markdown report with collapsible raw results

All data comes from the Orchestrator API (port 8000) — the UI contains no business logic.

Constants

```
ORCHESTRATOR_URL = "http://localhost:8000"
REQUEST_TIMEOUT = 120 # seconds
```

`REQUEST_TIMEOUT = 120` — the full research pipeline can take 30–60 seconds (LLM calls + web search + polling). 120 seconds gives generous headroom.

Page Configuration

```
st.set_page_config(
    page_title="Multi-Agent Research Assistant",
    page_icon="📄",
    layout="wide",
    initial_sidebar_state="expanded",
)
```

Must be the first Streamlit call. Streamlit raises an error if you call any other `st.*` function before `set_page_config()`. This sets the browser tab title, favicon, and layout mode.

`layout="wide"` — uses the full browser width instead of a centered column. Better for displaying long research reports.

Sidebar — Architecture Diagram

```

with st.sidebar:
    st.header("Architecture")
    st.code("""...""", language=None)

    st.header("Services")
    services = [
        ("Orchestrator", 8000, "LangGraph state machine + REST API"),
        ...
    ]
    for name, port, description in services:
        st.markdown(f"{name} · port {port}")
        st.caption(description)

```

`st.code(..., language=None)` — renders text in a monospace code block without syntax highlighting. Used for the ASCII architecture diagram.

`st.caption()` — smaller, greyed-out text. Good for secondary information.

The sidebar is visible at all times, showing the system architecture to anyone reviewing the demo.

Knowledge Base Upload

```

with st.expander("Upload document to Knowledge Base", expanded=False):
    uploaded_file = st.file_uploader(
        "Upload a PDF, TXT, or DOCX file",
        type=["pdf", "txt", "docx"],
    )

```

`st.expander(..., expanded=False)` — collapsible section. Collapsed by default so the upload UI doesn't clutter the main page.

`st.file_uploader(type=["pdf", "txt", "docx"])` — browser file picker that filters to the accepted types. Returns a `UploadedFile` object or `None`.

```

if uploaded_file is not None:
    if st.button("Add to Knowledge Base"):
        with st.spinner(f"Uploading {uploaded_file.name}..."):
            try:
                response = requests.post(

```



```

        f"{ORCHESTRATOR_URL}/upload",
        files={"file": (uploaded_file.name, uploaded_file.getvalue(), uploaded_file.type)},
        timeout=60,
    )
    response.raise_for_status()
    result = response.json()
    st.success(f"Uploaded **{uploaded_file.name}** – {result['num_chunks']} chunks indexed")

```

`uploaded_file.getvalue()` — returns the file content as bytes. Needed for the multipart POST.

`files={"file": (name, bytes, mime_type)}` — the `requests` library multipart format. This maps to what FastAPI's `UploadFile` expects.

`st.spinner()` — shows a spinning indicator while the upload is in progress. Streamlit blocks the UI during this time.

Two nested `if` checks:

- Outer `if uploaded_file is not None` — file has been selected
- Inner `if st.button(...)` — button has been clicked

This pattern prevents the upload from triggering automatically when a file is selected.

```

    except requests.exceptions.HTTPError:
        st.error(f"Upload failed: {response.json().get('detail', 'Unknown error')}")
    except Exception as exc:
        st.error(f"Upload failed: {exc}")

```

Two exception handlers:

- `HTTPError` — the API returned a 4xx/5xx. Extract the detail from the JSON body.
- General `Exception` — network errors, JSON parse failures, etc.

Live Document List

```

    try:
        docs_resp = requests.get(f"{ORCHESTRATOR_URL}/documents", timeout=5)
        if docs_resp.ok:
            docs = docs_resp.json().get("documents", [])
            if docs:
                st.caption(f"{len(docs)} document(s) in knowledge base:")

```

```

        for doc in docs:
            st.markdown(f"- **{doc['filename']}** - {doc['num_chunks']} chunks")
        else:
            st.caption("No documents uploaded yet.")
    except Exception:
        pass

```

Called every time the expander is open (Streamlit reruns the whole script on every interaction). The `try/except: pass` silently ignores errors — if the API is offline, the document list simply doesn't appear rather than crashing the upload UI.

`timeout=5` — short timeout since this is a status display, not a critical operation.

Research Form

```

with st.form(key="research_form"):
    query = st.text_input(
        label="Research query",
        placeholder="e.g. What are the latest advances in RAG systems?",
        help="...",
    )
    submitted = st.form_submit_button("Research", use_container_width=True)

```

`st.form` — groups widgets into a form. Streamlit doesn't rerun the script for widget changes inside a form; it only reruns when the submit button is clicked. Without this, every keystroke in the text input would trigger a rerun.

`use_container_width=True` — makes the button full-width.

Handling Submission

```

if submitted:
    query = query.strip()
    if not query:
        st.warning("Please enter a research query before clicking Research.")
        st.stop()

```

`st.stop()` — halts the script execution at this point. Equivalent to an early return.

```

with st.spinner("Agents researching..."):
    try:
        response = requests.post(
            f"{ORCHESTRATOR_URL}/research",
            json={"query": query},
            timeout=REQUEST_TIMEOUT,
        )
        response.raise_for_status()
        data = response.json()

```

`json={"query": query}` — `requests` automatically:

1. Serializes the dict to a JSON string
2. Sets `Content-Type: application/json`

The `with st.spinner()` block shows the spinner for the entire duration of the blocking `requests.post()` call.

Error Handling

```

except requests.exceptions.ConnectionError:
    st.error("Could not connect to the Orchestrator at ...")
    st.stop()

except requests.exceptions.Timeout:
    st.error(f"The request timed out after {REQUEST_TIMEOUT} seconds...")
    st.stop()

except requests.exceptions.HTTPError as exc:
    try:
        detail = response.json().get("detail", str(exc))
    except Exception:
        detail = str(exc)
    st.error(f"Orchestrator returned an error: {detail}")
    st.stop()

except Exception as exc:
    st.error(f"Unexpected error: {exc}")
    st.stop()

```

Four distinct error types with user-friendly messages:

- `ConnectionError` — orchestrator not running
- `Timeout` — pipeline took longer than 120 seconds
- `HTTPError` — API returned 4xx/5xx (extracts the detail field)
- Generic `Exception` — anything else

Each case calls `st.stop()` to prevent the results section from attempting to render with `data = None`.

Results Rendering

```
    final_report: str = data.get("final_report", "")
    web_results: str = data.get("web_results", "")
    rag_results: str = data.get("rag_results", "")
    status: str = data.get("status", "completed")

    st.success("Research complete!")

    if status and status != "completed":
        st.info(f"Pipeline status: `{status}`")

    st.markdown("## Final Report")
    if final_report:
        st.markdown(final_report)
    else:
        st.warning("The synthesis agent returned an empty report.")

    with st.expander("Web Search Results"):
        if web_results:
            st.text(web_results)
        else:
            st.caption("No web search results were returned.")

    with st.expander("Knowledge Base Results"):
        if rag_results:
            st.text(rag_results)
        else:
            st.caption("No knowledge base results were returned.")
```

`st.markdown(final_report)` — renders the markdown from the Synthesis Agent with proper headers, bullets, and bold text.

`st.text(web_results)` — renders pre-formatted plain text (preserves newlines and spacing). Used for raw agent output that's not meant to be markdown.

`st.expander()` for raw results — collapsed by default so the final report is the dominant element. Power users can expand to inspect what each agent returned.

Streamlit Execution Model

Every user interaction (button click, text change outside a form, expander toggle) causes Streamlit to **re-run the entire script from top to bottom**. This means:

- The document list fetch runs on every page load (acceptable with a 5s timeout)
- `submitted` is `True` only on the frame when the form button is clicked
- On the next interaction, `submitted` becomes `False` and the results disappear

This is why `st.session_state` would be needed to persist results across interactions in a production UI. For this demo, results are shown immediately after submission and the user re-submits if they want to run another query.

Section 12

Concepts

Concepts Glossary

Every concept used in this project, explained in one place.

Multi-Agent Systems

A pattern where multiple specialized AI agents collaborate to complete a task that would be too complex for a single agent. Each agent:

- Has a narrow, well-defined responsibility
- Communicates with other agents via a defined protocol
- Can be developed, tested, and replaced independently

In this project: The orchestrator delegates to web search, RAG, and synthesis agents. Each agent is a separate process with its own API.

Why not one big agent?

- Separation of concerns — each agent can be specialized for its task
 - Parallel execution — web search and RAG run simultaneously, cutting wall-clock time
 - Reliability — if one agent fails, only that step is retried, not the whole pipeline
 - Composability — agents can be reused in other pipelines
-

LangGraph

A Python library (part of the LangChain ecosystem) for building stateful multi-actor applications with LLMs. Key concepts:

StateGraph — a directed graph where nodes are Python functions and edges define execution flow. State is a TypedDict shared across all nodes.

Nodes — async Python functions that accept state and return a partial state update.

Edges — define what runs after each node. Can be:

- Fixed: `add_edge("node_a", "node_b")` — always goes to node_b
- Conditional: `add_conditional_edges("node_a", func, [...])` — func returns the next node

Send — LangGraph's parallel dispatch primitive. An edge function can return `[Send("node_x", state), Send("node_y", state)]` to run both nodes concurrently.

Fan-out / Fan-in — **Send** creates the fan-out (multiple nodes start). LangGraph automatically waits for all branches before advancing (fan-in).

`ainvoke()` — runs the compiled graph asynchronously, returns the final state.

Why LangGraph instead of plain asyncio?

LangGraph manages state, handles partial updates from parallel nodes, provides checkpointing (human-in-the-loop), and visualizes the graph. Plain asyncio would require manual state management and error handling.

A2A Protocol (Agent-to-Agent)

Google's 2025 draft standard for inter-agent HTTP communication. Three core endpoints:

```
GET /.well-known/agent.json    Agent Card – self-description
POST /tasks/send                Submit a task, returns task_id immediately
GET /tasks/{task_id}           Poll for task status + result
```

Agent Card — a JSON document (inspired by OpenAPI) at the well-known URL that describes the agent's name, capabilities, and version. Enables agent discovery.

Task lifecycle: `submitted` → `working` → `completed` | `failed`

Why submit-then-poll instead of blocking POST?

LLM operations take 5–30 seconds. A blocking POST that long is fragile. Submit-then-poll lets the server return quickly and the client poll asynchronously.

In this project: We implement A2A manually with FastAPI — no external SDK needed. The protocol is just JSON-RPC 2.0 over HTTP.

MCP (Model Context Protocol)

Anthropic's 2024 open standard for connecting LLM clients to external tools and data. Analogous to USB for AI tools — any MCP client connects to any MCP server.

Transport: stdio (local subprocess), HTTP+SSE (remote), WebSocket.

Protocol:

1. Client sends `list_tools()` — server returns tool definitions with JSON schemas
2. Client sends `call_tool(name, args)` — server runs the tool and returns `TextContent`

Tool schema — a JSON Schema object describing the tool's input parameters.

Claude (and other LLMs) use this to know what arguments to provide.

In this project: The MCP server exposes rag-doc-qa tools to Claude Desktop. The main research pipeline does NOT use MCP at runtime (the RAG agent imports directly).

TypedDict

A Python type from the `typing` module. A dict where every key has a declared type.

Used instead of a Pydantic model for LangGraph state because:

- Nodes return partial dicts (only changed keys) — LangGraph merges them
- TypedDict allows this naturally; Pydantic models would require returning all fields

```
class ResearchState(TypedDict):  
    query: str  
    web_results: str  
    # ...
```

At runtime, `ResearchState` is just a regular dict. The `TypedDict` annotation only affects static type checkers (mypy, pyright) and LangGraph's internal validation.

`asyncio.to_thread()`

Runs a synchronous (blocking) function in a thread pool executor without blocking the asyncio event loop.

```
result = await asyncio.to_thread(blocking_function, arg1, arg2)
```

Why this matters: Python's asyncio event loop is single-threaded. If you call a blocking function directly inside an `async def`, you block the entire event loop — no other coroutines can run until it returns. `asyncio.to_thread()` offloads the blocking work to a separate thread, freeing the event loop.

Used for: ChromaDB queries, Anthropic SDK calls, document processing — any function that makes synchronous network calls or disk I/O.

Alternative: `asyncio.get_event_loop().run_in_executor(None, func, *args)` — older equivalent. `asyncio.to_thread()` is the modern (Python 3.9+) shorthand.

`asyncio.create_task()`

Schedules a coroutine to run on the event loop without waiting for it.

```
asyncio.create_task(_run())    # starts _run() in background, returns immediately
```

Used in: `base_a2a_server.py` — the POST `/tasks/send` endpoint fires off the background task with `create_task()` and returns the `task_id` immediately.

vs `await coroutine()`: `await` blocks until the coroutine completes.
`create_task()` schedules it and continues executing immediately.

httpx

A Python HTTP client with both sync and async interfaces. Used instead of `requests` in the A2A client because:

- `requests` is synchronous — calling it in async code blocks the event loop
- `httpx.AsyncClient` is natively async — `await client.get(...)` yields to the event loop
- Similar API to `requests` — easy to learn

`httpx.Timeout` — allows different timeouts for connect, read, write, and pool phases of an HTTP request.

Tavily API

A web search API built specifically for LLM applications. Returns pre-cleaned, snippet-ready results — no HTML parsing required. Free tier: 1000 searches/month.

`search_depth="basic"` — faster and cheaper, returns snippets from search result pages. `"advanced"` fetches and parses full page content.

RAG (Retrieval-Augmented Generation)

A pattern where relevant documents are retrieved and given to an LLM as context for answering a question. Prevents hallucination by grounding the answer in source material.

Full explanation in [rag-doc-qa docs](#).

Hybrid Search

Combining semantic (embedding-based) search with keyword (BM25) search. Results from both are merged using Reciprocal Rank Fusion. See rag-doc-qa docs for full details.

Structured Outputs

Getting an LLM to return machine-parseable JSON instead of free-form text.

The `plan_node` uses this: the system prompt instructs Claude to return `{"web_search_query": ..., "rag_query": ...}` and nothing else. The response is parsed with `json.loads()`.

Challenges:

- LLMs sometimes add markdown code fences (````json ... ````) — must strip them
 - Field names must match exactly — the prompt specifies the exact JSON structure
-

Microservices Pattern

Breaking an application into small, independently deployable services. Each service in this project:

- Has a single responsibility (web search, RAG, synthesis)
- Communicates over HTTP (A2A protocol)
- Can be replaced without affecting other services
- Runs as its own process on its own port

Trade-off: More complex to set up (5 terminals/processes) vs. simpler monolith. Worth it here because it demonstrates real-world production patterns.

Factory Function Pattern

A function that creates and returns another object (typically a class instance or configured object). `create_a2a_app()` is a factory — it takes configuration parameters and returns a fully configured FastAPI app.

Advantages over a base class:

- Simpler — no class hierarchy
 - More flexible — behavior is injected via `process_task` callable
 - Each call creates an isolated instance with its own task store (closure)
-

Closure

A function that "closes over" variables from its enclosing scope. In `create_a2a_app()`:

```
def create_a2a_app(...):
    tasks = {} # ← this variable

    @app.post("/tasks/send")
    async def send_task(request):
        tasks[new_id] = ... # ← is captured by these inner functions
```

The inner endpoint functions (`send_task`, `get_task`) capture `tasks` from the outer scope. Each call to `create_a2a_app()` creates a new `tasks` dict that is independent

for each agent.

Pydantic `Field`

A Pydantic utility for adding metadata to model fields:

```
query: str = Field(..., min_length=3, max_length=2000, description="...", examples=["..."])
```

- `...` (Ellipsis) — required field (no default)
- `min_length`, `max_length` — validation constraints
- `description` — shown in FastAPI's auto-generated Swagger UI
- `examples` — sample values shown in Swagger UI

`/.well-known/` URL Path

RFC 8615 defines `/.well-known/` as a standardised namespace for service-level metadata URLs. Examples:

- `/.well-known/agent.json` — A2A Agent Card (this project)
- `/.well-known/openid-configuration` — OAuth/OIDC server metadata
- `/.well-known/security.txt` — security disclosure contact

Using the standard path means any A2A-aware tool knows exactly where to find agent metadata without documentation.

`ensure_ascii=False` in `json.dumps()`

By default, `json.dumps()` escapes all non-ASCII characters:

```
json.dumps({"text": "café"}) → '{"text": "caf\\u00e9"}'
```

With `ensure_ascii=False`:

```
json.dumps({"text": "café"}, ensure_ascii=False) → '{"text": "café"}'
```

Used throughout this project when JSON payloads may contain non-English text from web search results or uploaded documents. The resulting strings are more readable and slightly smaller.

Section 13

Interview Prep

Interview Prep — Multi-Agent Research Assistant

25 questions covering LangGraph, A2A, MCP, async Python, and system design.

Every answer is grounded in this project's actual code.

LangGraph

Q1: What is LangGraph and why did you use it instead of plain asyncio?

LangGraph is a library for building stateful multi-actor applications. I used it because it provides:

- A TypedDict-based state that all nodes share and update
- First-class parallel fan-out with the `Send` API
- Automatic fan-in — it waits for all parallel branches before continuing
- Conditional edges for routing decisions

With plain asyncio I'd need to manually manage the shared state dict, handle concurrent writes, and implement the fan-in with `asyncio.gather()`. LangGraph handles all of that.

Q2: How does the parallel fan-out work in your LangGraph graph?

After `plan_node` finishes, I use `add_conditional_edges()` with a function that returns a list of `Send` objects:

```
def dispatch_parallel_searches(state):  
    return [  
        Send("web_search_node", state),  
        Send("rag_search_node", state),  
    ]
```

Each `Send` tells LangGraph to schedule that node with the given state snapshot. LangGraph runs both concurrently in the asyncio event loop. After both complete, it merges their partial state updates before advancing to `synthesize_node`.

Q3: Why do your parallel nodes not update the `status` field?

If both `web_search_node` and `rag_search_node` return `{"status": "..."} simultaneously, LangGraph sees two concurrent writes to the same state key and raises INVALID_CONCURRENT_GRAPH_UPDATE. Each parallel node must write to different keys. I resolved this by removing status from the parallel nodes' return dicts — only plan_node and synthesize_node update it.`

Q4: What's the difference between `add_edge` and `add_conditional_edges`?

- `add_edge("a", "b")` — always routes from node a to node b
- `add_conditional_edges("a", func, [possible_nodes])` — calls `func(state)` after node a, which can return a node name (string) OR a list of `Send` objects for parallel dispatch. The third argument declares valid destinations for graph validation.

Q5: What does `graph.ainvoke(initial_state)` return?

The final `ResearchState` dict after all nodes have executed. It contains the fully populated state: `final_report`, `web_results`, `rag_results`, `status`, etc.
`ainvoke` is the async version of `invoke` — it must be awaited in an async context.

A2A Protocol

Q6: What is the A2A protocol and why use it for inter-service communication?

A2A (Agent-to-Agent) is Google's 2025 draft standard for HTTP communication between AI agents. Three endpoints: Agent Card at `/.well-known/agent.json`, task submission at `POST /tasks/send`, and polling at `GET /tasks/{task_id}`.

I used it because:

- It's a real emerging standard — shows awareness of the ecosystem
- The submit-then-poll pattern handles long-running LLM operations gracefully
- Each agent is independently discoverable via its Agent Card
- Standard protocol means any agent can be swapped without changing the orchestrator

Q7: Why does the A2A task submission return immediately rather than blocking?

LLM operations (web search + parsing, ChromaDB + Claude) take 10–30 seconds. A blocking POST of that duration would tie up HTTP connections and is fragile. The submit-then-poll pattern lets the server respond immediately (task submitted) and the client check progress asynchronously. This is also more resilient — if the client disconnects and reconnects, it can resume polling.

Q8: How did you implement the A2A server without an external SDK?

With FastAPI. The `base_a2a_server.py` factory creates a FastAPI app with three endpoints. `POST /tasks/send` uses `asyncio.create_task()` to run `process_task()` in the background without blocking the response. Task state is stored in an in-memory

dict. The `GET /tasks/{task_id}` endpoint looks up status from that dict.

Q9: What happens if an agent fails while processing a task?

The inner `_run()` coroutine in `base_a2a_server.py` has a broad `except Exception` handler. Any failure sets `tasks[task_id]["status"] = "failed"` and stores the error message in `result`. When the orchestrator polls and sees `status == "failed"`, it raises `A2AError`, which propagates to `api/main.py` and becomes an HTTP 500 response.

MCP Protocol

Q10: What is MCP and what does your MCP server expose?

Model Context Protocol is Anthropic's 2024 open standard for connecting LLM clients to tools. My MCP server exposes three tools:

- `search_documents(query, k)` — hybrid search over the knowledge base
- `answer_with_rag(question)` — full RAG answer with citations
- `list_documents()` — enumerate indexed documents

It uses stdio transport, so Claude Desktop can spawn it as a subprocess.

Q11: Does the main pipeline use the MCP server?

No — the MCP server is a standalone tool for connecting Claude Desktop to the same knowledge base. The RAG Agent imports `rag-doc-qa` directly via `sys.path` injection. I built the MCP server to demonstrate the protocol; the direct import is simpler and faster for the pipeline.

Q12: Why does the MCP server log to stderr instead of stdout?

The MCP stdio transport uses stdout for the JSON protocol messages. Any text written to stdout would corrupt the protocol framing. Logging must go to stderr.

Async Python

Q13: Why do you use `asyncio.to_thread()` in multiple places?

The Anthropic Python SDK, ChromaDB, and `process_upload` are all synchronous. Calling them directly inside an `async def` would block the asyncio event loop — no other coroutines could run while they execute. `asyncio.to_thread()` runs them in a thread pool, so the event loop stays responsive.

Q14: What's the difference between `asyncio.create_task()` and `await coroutine()`?

- `await coroutine()` — suspends the current coroutine until the awaited one completes
- `asyncio.create_task(coroutine())` — schedules the coroutine to run concurrently and returns immediately

Used in `base_a2a_server.py`: we `create_task(_run())` so the POST endpoint returns the `task_id` without waiting for the task to finish.

Q15: What would happen if you called `time.sleep(1)` instead of `await asyncio.sleep(1)` in the A2A polling loop?

`time.sleep(1)` blocks the entire OS thread — the asyncio event loop cannot run any other coroutines during that second. If both `web_search_node` and `rag_search_node` are polling simultaneously, they'd each block for 1 second alternately rather than truly running in parallel. The fix is `await asyncio.sleep(1)`, which suspends only the current coroutine while the event loop processes other work.

System Design

Q16: How does the orchestrator pass context to the synthesis agent?

`synthesize_node` builds a JSON string:

```
synthesis_input = json.dumps({
    "query": state["query"],
    "web_results": state["web_results"],
    "rag_results": state["rag_results"],
})
```

This JSON is sent as the `message` field in the A2A task request. The Synthesis Agent parses it with `json.loads()`. Using JSON (rather than concatenated plain text) ensures the three fields are cleanly separated and parseable.

Q17: Why does each agent do `sys.path.insert(0, str(_PROJECT_ROOT))`?

Python resolves imports relative to `sys.path`. When you run `python agents/web_search/agent.py`, the working directory may not be `multi-agent-researcher/`, so `from agents.base_a2a_server import ...` would fail. Inserting `_PROJECT_ROOT` at position 0 ensures it's checked first.

Q18: Why use `sys.path` injection to reuse rag-doc-qa instead of copying the code?

Copying would create two independent codebases that can drift. More importantly, the RAG pipeline uses a shared `chroma_db/` directory — copying the code would still require a shared database. The `sys.path` approach means:

- One codebase, zero duplication
- One ChromaDB — documents uploaded via the orchestrator are immediately available
- Changes to rag-doc-qa (new models, prompt tweaks) automatically apply to this project

Q19: How would you scale this system if it needed to handle 100 concurrent research requests?

- Replace the in-memory task store with Redis
- Deploy each agent as a horizontally scaled service (multiple instances behind a load balancer)
- Use a proper task queue (Celery, RQ, or Cloud Tasks) instead of `asyncio.create_task()`
- Add database persistence for LangGraph state (LangGraph has built-in PostgreSQL checkpointing)
- Use `uvicorn --workers N` for the FastAPI services

Q20: What would you change to add a new specialist agent (e.g., a code search agent)?

1. Create `agents/code_search/agent.py` with a `process_task()` function and call `create_a2a_app()`
2. Add `code_search_results: str` to `ResearchState` in `state.py`
3. Add `code_search_node` to `graph.py`
4. Add another `Send("code_search_node", state)` to `dispatch_parallel_searches()`
5. Update `synthesize_node` to include `code_search_results` in the payload

Everything else (the A2A client, base server, UI) works unchanged.

Debugging & Errors

Q21: You encountered `INVALID_CONCURRENT_GRAPH_UPDATE`. What caused it and how did you fix it?

Both `web_search_node` and `rag_search_node` were returning `{"status": "web_searching"}` and `{"status": "rag_searching"}` simultaneously. LangGraph tried to merge both updates into the `status` field and detected a conflict. Fix: removed `status` from both parallel nodes' return dicts. Only the sequential nodes (`plan_node`, `synthesize_node`) update it.

Q22: You hit `httpx.ReadTimeout` on the synthesis POST. What caused it and how did you fix it?

The Synthesis Agent's `process_task()` called `client.messages.create()` (synchronous Anthropic SDK) directly inside an `async def`. This blocked the asyncio event loop for 10–15 seconds — the event loop couldn't send the HTTP response back to the A2A client,

causing a read timeout. Fix: wrapped the Claude call in `asyncio.to_thread(_call_claude)`.

Q23: What's the Pydantic `"extra": "ignore"` fix you needed for rag-doc-qa?

rag-doc-qa's `Settings` class (Pydantic `BaseSettings`) had strict field validation. When `TAVILY_API_KEY` was in the environment (loaded from multi-agent-researcher's `.env`), Pydantic rejected it as an unexpected field. Fix: added `"extra": "ignore"` to rag-doc-qa's `model_config`, telling Pydantic to silently ignore unknown environment variables.

Design Decisions

Q24: Why did you choose `claude-haiku-4-5-20251001` for the planner and `claude-sonnet-4-6` for synthesis?

The planner task is simple: decompose a question into two sub-queries and return valid JSON. Haiku is 3× faster and much cheaper than Sonnet, and JSON generation is well within its capability.

Synthesis requires reading two large result sets, identifying contradictions, and writing a structured 400-word report — a genuine reasoning task where Sonnet's quality advantage is worth the cost.

Q25: The `chroma_db/` folder is gitignored. How do users get it populated?

The database is populated by uploading documents through the UI. It's gitignored because:

- Binary files don't diff meaningfully in git
- The database is machine-specific (paths, embeddings model version)
- Users will want their own documents, not test documents from the developer

Users upload files via the Streamlit "Upload document to Knowledge Base" expander, which calls `POST /upload` on the orchestrator, which calls rag-doc-qa's `process_upload()` to chunk, embed, and index the document into ChromaDB.